Kubernetes — এক নজরে

Kubernetes কী

Kubernetes (ছোট করে **K8s**) একটা open-source system যেটা অনেকগুলো server জুড়ে container চালায়, নজরে রাখে, আর কিছু ভাঙলে নিজে থেকে ঠিক করে। আপনি শুধু লিখে দেন **"কী চাই"** — যেমন "আমার API এর ৩টা কপি সবসময় চলবে" — আর Kubernetes সারাক্ষণ মিলিয়ে দেখে বাস্তবে সেটা আছে কিনা। না থাকলে নিজেই ঠিক করে।

উদাহরণ — thermostat: AC তে 24°C সেট করলে সে বারবার ঘরের তাপমাত্রা মাপে আর নিজে ঠান্ডা বা গরম করে। আপনাকে প্রতি ঘণ্টায় হাত দিতে হয় না। Kubernetes ও এভাবে কাজ করে: আপনার লেখা **desired state** (যা চান) আর **actual state** (যা চলছে) সবসময় মেলায়। এই অবিরাম মেলানোকে বলে **reconciliation**, আর পুরো Kubernetes এর মূল ধারণা এটাই।

Google এর ভেতরের system থেকে অনুপ্রাণিত হয়ে ২০১৪ সালে এটা open-source হয়। এখন CNCF এর অধীনে চলে, আর AWS (EKS), Google (GKE), Azure (AKS) সহ প্রায় সব cloud এ managed সেবা হিসেবে পাওয়া যায়।

কোন সমস্যা সমাধান করে

একটা server এ একটা app চালানো সহজ; সমস্যা শুরু হয় যখন অনেক app, অনেক server, আর রাত ৩টায় কিছু ভেঙে যায়। Kubernetes এই "অনেক কিছু একসাথে সামলানো" কাজটা (**orchestration**) automate করে।

পরিস্থিতি	Kubernetes ছাড়া	Kubernetes সহ
App crash করল	কেউ alert দেখে SSH করে restart করে	নিজে থেকে restart (**self-healing**)
পুরো server মারা গেল	ওই server এর সব app বন্ধ, হাতে অন্য server এ সরাতে হয়	২–৫ মিনিটে অন্য server এ নতুন কপি চালু
Traffic হঠাৎ বেড়ে গেল	হাতে নতুন server আর app কপি যোগ	CPU দেখে নিজে কপি বাড়ায় (**autoscaling**)
নতুন version deploy	app বন্ধ → নতুন চালু → কিছুক্ষণ downtime	একটা একটা করে কপি বদলায় (**rolling update**), downtime ছাড়া

পরিস্থিতি	Kubernetes ছাড়া	Kubernetes সহ
নতুন version এ bug	পুরনো version খুঁজে আবার deploy	এক command এ **rollback**
কোন app কোন server এ রাখব	মানুষ হিসাব করে, server খালি পড়ে থাকে বা ভরে যায়	**scheduler** খালি জায়গা দেখে নিজে বসায়
App খুঁজে পাওয়া	IP hardcode, server বদলালে config বদলাও	নাম দিয়ে খোঁজা (**service discovery**), IP নিয়ে চিন্তা নেই
Config আর secret	server এ server এ ছড়ানো `.env` file	এক জায়গায় ConfigMap আর Secret

মূল শব্দগুলো

এই ১৫টা শব্দ জানলেই Kubernetes এর যেকোনো আলোচনা বোঝা যায়; ছোট থেকে বড় ক্রমে সাজানো।

শব্দ	সহজ মানে	উদাহরণ	কোন module
Container	app আর তার সব দরকারি জিনিস একসাথে প্যাক করা, আলাদা করে চালানো একটা process	Node.js API একটা container এ চলে	M0.4
Image	container এর ছাঁচ — read-only, একবার বানিয়ে বারবার চালানো যায়	`subtrack-api:1.0.0`	M0.4
Registry	image রাখার দোকান	Docker Hub, GHCR, ECR	M0.4
Node	একটা machine (server বা VM) যেখানে container চলে	আমাদের lab এ ৩টা node	M1.2
Control plane	cluster এর মস্তিষ্ক — সিদ্ধান্ত নেয়, নজর রাখে	API server, etcd, scheduler, controller-manager	M1.2
Worker node	যে node এ আপনার app আসলে চলে	`subtrack-lab-worker`	M1.2
Cluster	control plane + সব worker node মিলে পুরো ব্যবস্থা	`kind-subtrack-lab`	M1.2

শব্দ	সহজ মানে	উদাহরণ	কোন module
Pod	Kubernetes এর সবচেয়ে ছোট একক — এক বা একাধিক container এর একটা মোড়ক, নিজস্ব IP সহ	API এর একটা কপি = একটা Pod	M2.1
Deployment	"এই Pod এর N টা কপি সবসময় রাখো, আর version বদলাও নিরাপদে"	`api` Deployment, ৩ replica	M2.2
Service	কয়েকটা Pod এর সামনে একটা স্থায়ী নাম আর IP — Pod বদলালেও ঠিকানা একই	`http://api`	M3.1
Namespace	cluster এর ভেতরে আলাদা ঘর	`subtrack-dev`, `subtrack-prod`	M2.4
ConfigMap / Secret	config আর গোপন তথ্য (password, key) code থেকে আলাদা রাখা	`DATABASE_URL`	M3.4, M3.5
YAML / manifest	যে file এ "কী চাই" লেখা হয়	`deployment.yaml`	M0.5, M1.6
kubectl	cluster কে নির্দেশ দেওয়ার command line tool	`kubectl apply -f deployment.yaml`	M1.5
Desired state	YAML এ যা লিখেছেন — Kubernetes সবসময় বাস্তবকে এর সাথে মেলায়	"৩টা Pod" লিখলে ৩টাই রাখবে	M1.1

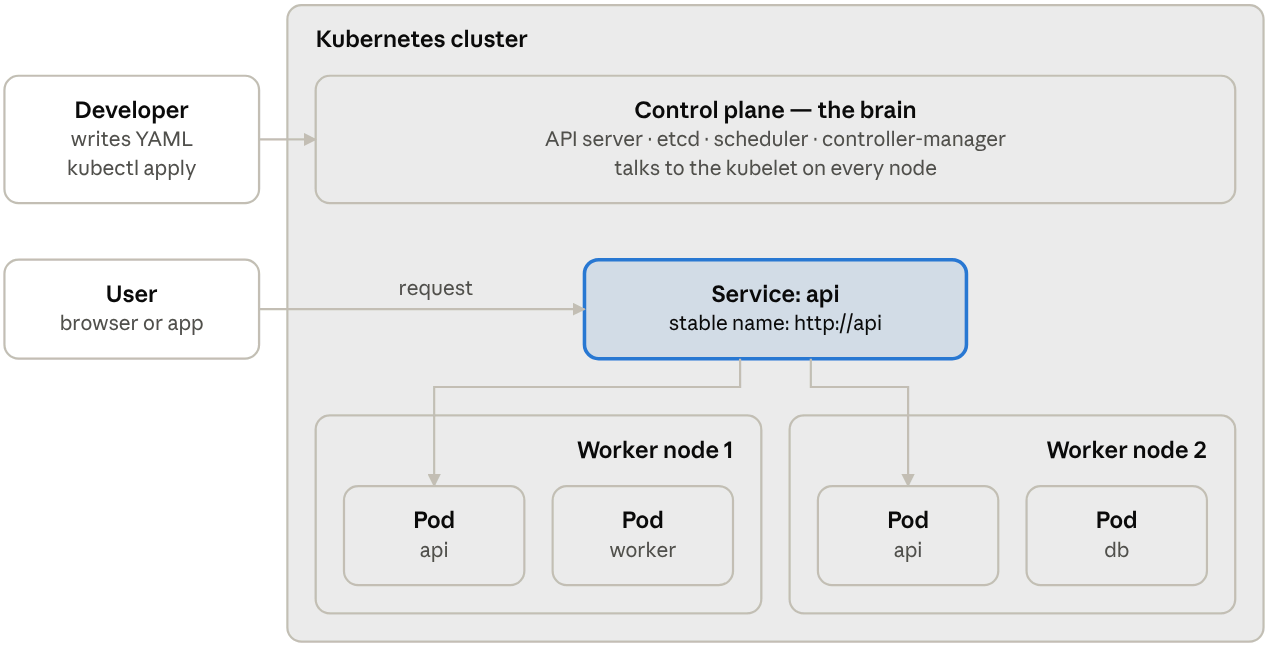
একটা YAML দেখতে এরকম — "`api` এর ৩টা কপি চালাও":

```
apiVersion: apps/v1          # কোন API version
kind: Deployment             # কী জিনিস বানাচ্ছি
metadata:
  name: api                  # নাম
spec:                        # কী চাই (desired state)
  replicas: 3                # ৩টা কপি
  selector:
    matchLabels: { app: api }
  template:                  # প্রতিটা Pod দেখতে কেমন
    metadata:
      labels: { app: api }
    spec:
      containers:
      - name: api
        image: subtrack-api:1.0.0
        ports:
        - containerPort: 3000
```

Cluster এর ভেতরে কী আছে

Cluster দুই ভাগে: **control plane** সিদ্ধান্ত নেয় আর নজর রাখে, **worker node** এ আপনার app এর Pod চলে। User এর request আসে **Service** এর মধ্য দিয়ে।

Users reach Pods through a Service; the control plane only manages

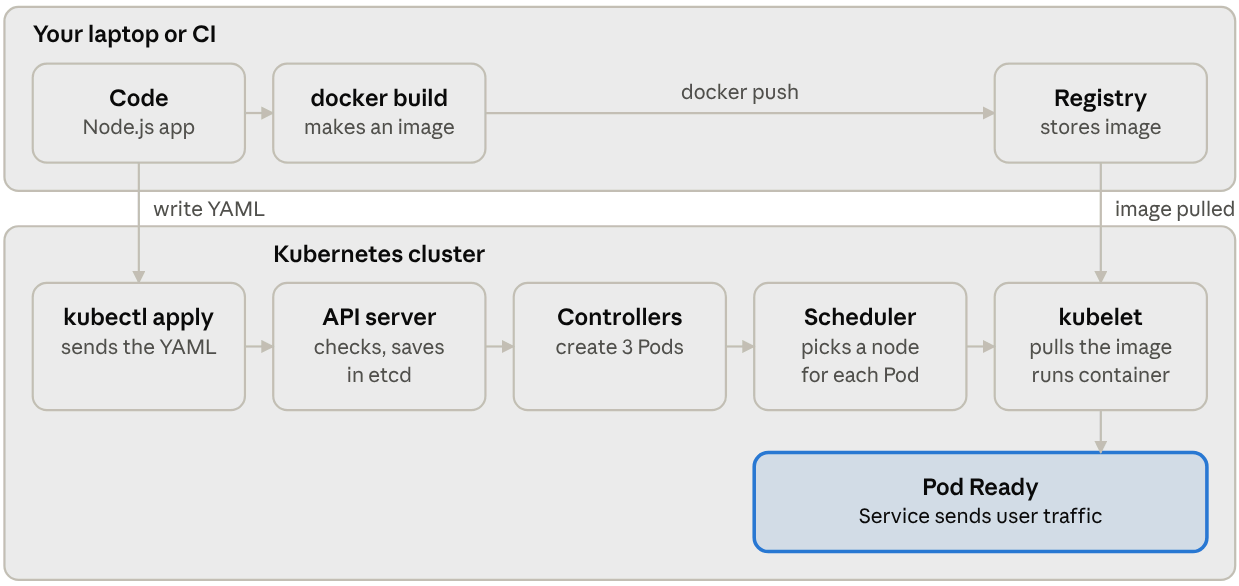


Cluster এর ভেতর · control plane, Service, ২টা worker node

User এর request সবসময় Service হয়ে কোনো একটা সুস্থ `api` Pod এ যায় — কোন Pod কোন node এ আছে, user এর জানার দরকার নেই। Control plane user traffic ছোঁয় না; সে শুধু নজর রাখে আর ঠিক করে। একটা worker node মারা গেলে তার Pod অন্য node এ নতুন করে চালু হয়, আর Service নিজে থেকে নতুনটায় traffic পাঠায়।

Code থেকে চলমান app

আপনি শুধু image বানান আর YAML পাঠান; বাকি সব কাজ cluster এর আলাদা আলাদা component নিজে থেকে করে।



Deploy flow · code থেকে চলমান Pod

`kubectl apply` এর পর API server YAML যাচাই করে etcd তে রাখে → controller দেখে ৩টা Pod দরকার, বানায় → scheduler প্রতিটা Pod এর জন্য node বাছে → সেই node এর kubelet registry থেকে image এনে container চালায় → Pod Ready হলে Service তাতে traffic পাঠায়। এরপরেও মেলানো থামে না: container crash করলে kubelet restart করে, Pod হারিয়ে গেলে controller নতুন বানায়।

Use case আর trade-off

Kubernetes শক্তিশালী কিন্তু ভারী — অনেকগুলো service আর traffic ওঠানামা থাকলে লাভ বেশি, একটা ছোট app এ খরচ বেশি।

কোথায় ভালো খাটে

Use case	কেন Kubernetes
অনেকগুলো service (microservices)	প্রতিটা আলাদা deploy, scale, rollback; একে অপরকে নাম দিয়ে খোঁজে
Traffic ওঠানামা করে (campaign, অফার, রাতের spike)	autoscaling — লোড বাড়লে কপি বাড়ে, কমলে কমে
Downtime চলে না (payment, callback, webhook)	rolling update, self-healing, একাধিক কপি

Use case	কেন Kubernetes
ঘনঘন release	CI/CD থেকে এক command এ deploy, সমস্যা হলে rollback
Background job আর schedule	Job (একবার), CronJob (নির্দিষ্ট সময়ে — যেমন renewal)
একাধিক environment (dev/staging/prod)	একই YAML, আলাদা namespace বা cluster
Cloud বদলানোর স্বাধীনতা	একই YAML AWS, Google, Azure বা নিজের server এ চলে

Trade-off

লাভ	দাম
Self-healing, autoscaling, zero-downtime	শেখার পথ লম্বা — নতুন অনেক ধারণা
সব কিছু YAML এ, Git এ রাখা যায়	YAML অনেক, ভুল indentation বা label এ নীরব bug
একই ভাবে সব cloud এ	Cluster চালানো নিজেই একটা কাজ — upgrade, monitoring, security
Resource ভালো ব্যবহার	ছোট setup এ control plane আর tools এর জন্য বাড়তি খরচ
বড় ecosystem (Helm, Argo CD, Prometheus)	Debug করা কঠিন — সমস্যা app এ, network এ, নাকি cluster এ?

কখন Kubernetes **নেবেন না**

- একটা বা দুটো ছোট app, স্থির traffic → একটা VM বা managed PaaS (যেমন Render, Railway, Cloud Run) সহজ ও সস্তা।
- Team এ কেউ cluster চালাতে পারে না, আর managed Kubernetes ও নেওয়া সম্ভব না।
- Database চালানোর জন্য একা → managed database (RDS, Cloud SQL) প্রায় সবসময় নিরাপদ।

সৎ নিয়ম: **সমস্যাটা আগে থাকতে হবে।** "সবাই ব্যবহার করে" — এটা কারণ না।

আমাদের কেন শেখা দরকার

Payment আর subscription এর মতো কাজে downtime মানে টাকা হারানো, আর একাধিক product এর backend একসাথে সামলাতে হয় — ঠিক এই দুই জায়গাতেই Kubernetes সবচেয়ে বেশি কাজে আসে।

- **একাধিক product, একটা shared payment gateway wrapper:** প্রতিটা product এর backend আলাদা service, wrapper আরেকটা — প্রতিটা আলাদা deploy আর scale, নাম দিয়ে একে অপরকে খোঁজে।
- **Callback আর webhook হারানো চলবে না:** একাধিক কপি, rolling update আর graceful shutdown মিলে deploy এর সময়ও request হারায় না।
- **Campaign বা অফারের সময় traffic spike:** autoscaling।
- **Renewal আর reconciliation এর মতো নির্দিষ্ট সময়ের কাজ:** CronJob।
- **Developer হিসেবেও দরকার:** platform team cluster চালালেও নিজের app এর YAML, probe, resource আর debug আমাদেরই করতে হয়। CrashLoopBackOff এর মানে না বুঝলে নিজের app এর সমস্যাই ধরা যায় না।

এই doc আর roadmap

এখানে যা দেখলেন	Roadmap এ বিস্তারিত
Container, image, node	P0 (M0.4), P1 (M1.2)
Control plane, reconciliation, deploy flow	P1 (M1.1–M1.3)
Pod, Deployment, rolling update, rollback	P2
Service, DNS, ConfigMap, Secret	P3
Storage, database	P4
Self-healing, autoscaling, probe	P5
Job, CronJob	P7
Security	P8
CI/CD, Helm, GitOps	P9
Debugging, monitoring	P10

পুরো পরিকল্পনা: Kubernetes Zero-to-Advanced Roadmap।

Team presentation outline (১০ মিনিট)

মূল বার্তা: "Kubernetes আমাদের app কে নিজে থেকে চালু রাখে, বাড়ায় আর নিরাপদে update করে — আমরা শুধু লিখি কী চাই।"

1. **সমস্যা দিয়ে শুরু (২ মিনিট):** "রাত ৩টায় payment callback server crash করলে কী হয়?" — "কোন সমস্যা সমাধান করে" table এর প্রথম তিন লাইন।
2. **এক ধারণা (১ মিনিট):** thermostat — desired state বনাম actual state।
3. **ছবি ১ (৩ মিনিট):** cluster এর ভেতর — user এর request কীভাবে Service হয়ে Pod এ পৌঁছায়, control plane কী করে।
4. **ছবি ২ (২ মিনিট):** code থেকে চলমান app — `kubectl apply` এর পর কী কী ঘটে।
5. **সৎ trade-off (১ মিনিট):** কখন নেব না।
6. **প্রশ্ন (১ মিনিট):** "একটা worker node মারা গেলে কী হবে?" — শ্রোতারা ছবি ১ দেখে উত্তর দিক।